FPT Education

TRƯỜNG ĐẠI HỌC FPT

Module 11 - Testing & Debugging in Flutter

Learning Objectives

Content:

- Identify common issues in Flutter apps
- Test logic, UI behavior, and navigation
- Validate complete user flows
- Use DevTools for debugging UI and performance
- Apply these techniques in Lab 11

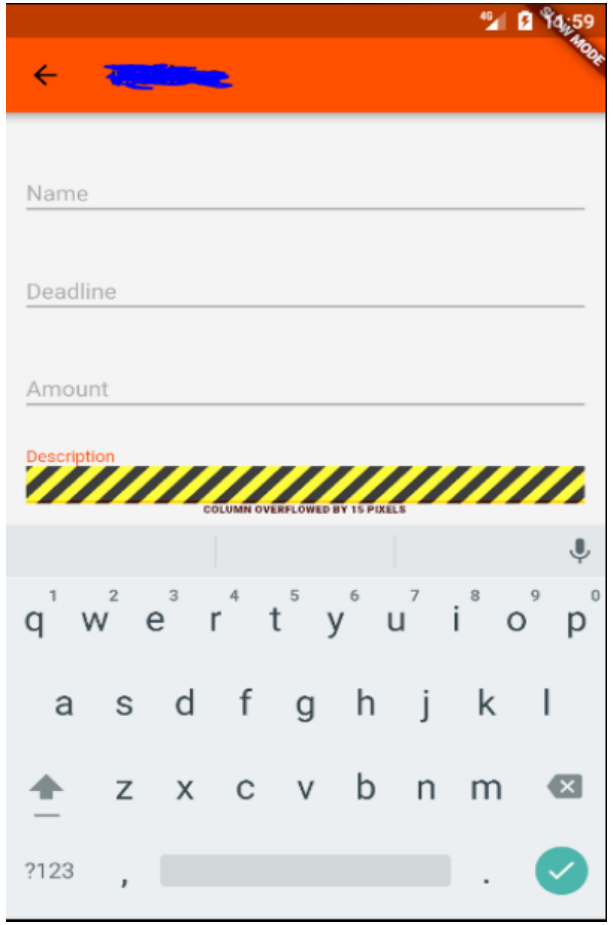
Why Performance Matters

- Users expect smooth 60–120 FPS experiences
- Poor rebuild patterns increase CPU load
- Large assets slow startup times
- Inefficient list rendering affects scrolling
- App size impacts downloads & Play Store acceptance

Example of a Flutter UI Problem

UI Overflow Error Example

- Occurs when a widget exceeds its layout constraints.
- Explain that these issues do not crash the app, so tests or DevTools are needed to detect them.

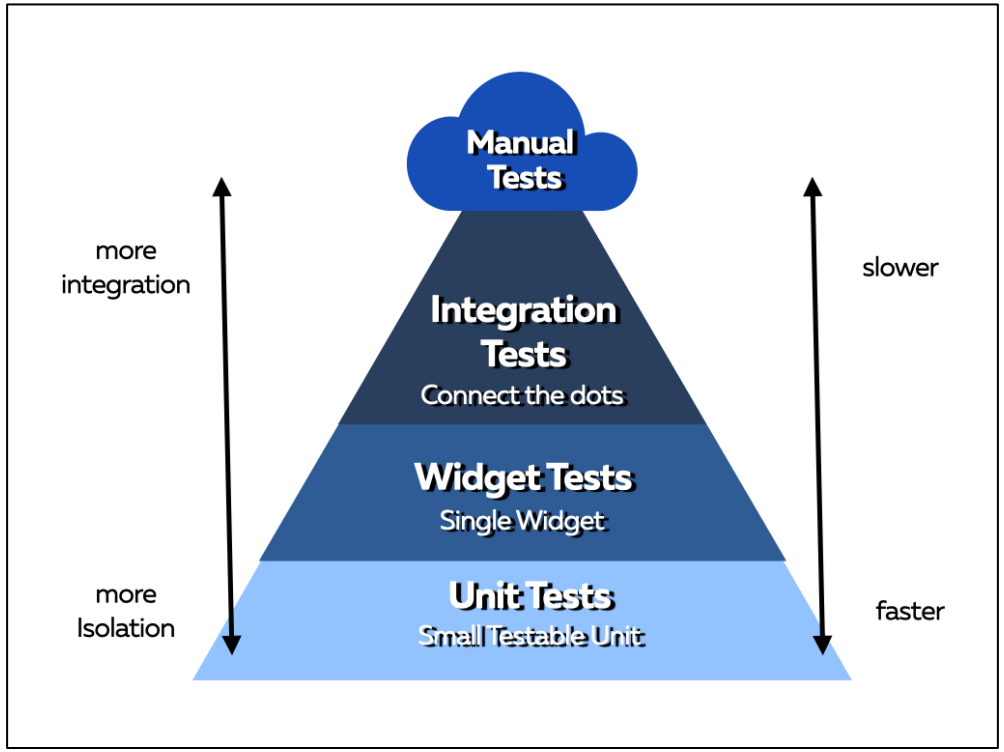


Types of Tests in Flutter

Three essential test types:

1. Unit Tests
   - Validate logic without UI
2. Widget Tests
   - Validate UI rendering & interaction
3. Integration Tests
   - Validate full user flows from start to finish

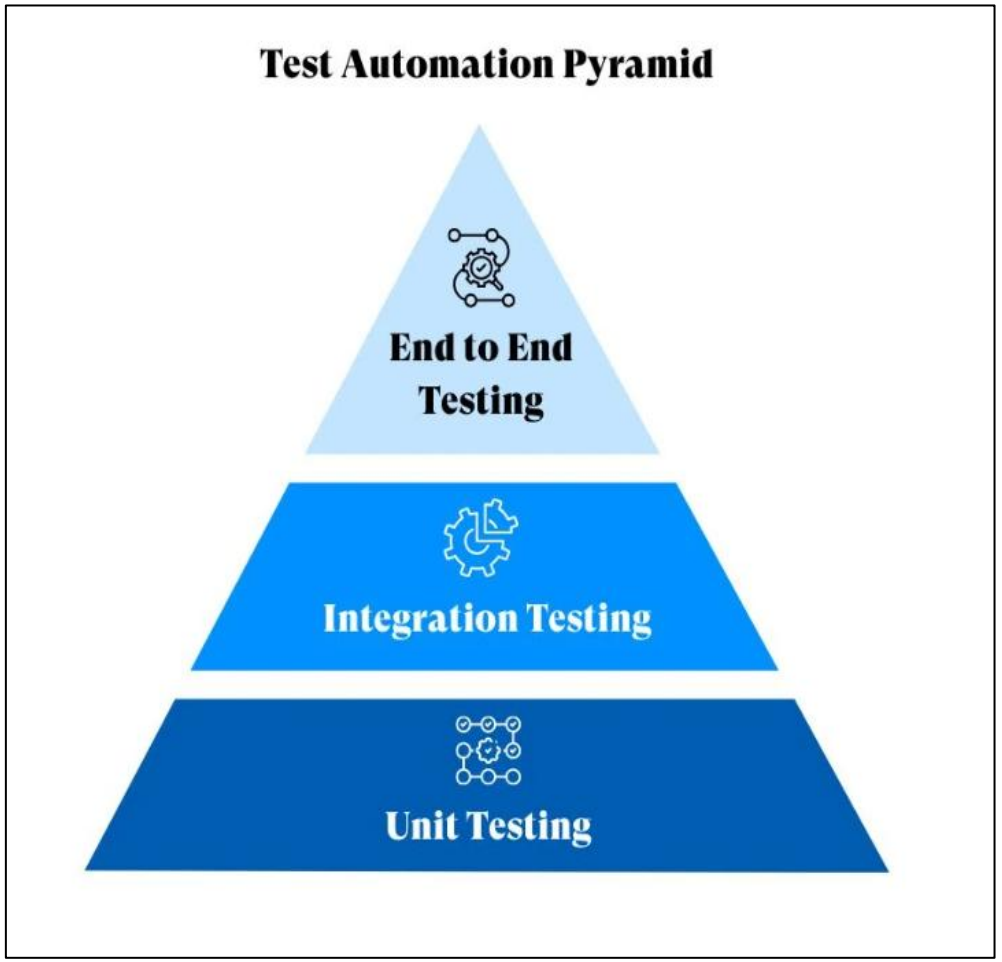
Layered Architecture Diagram



The Flutter Testing Pyramid

Why It Matters

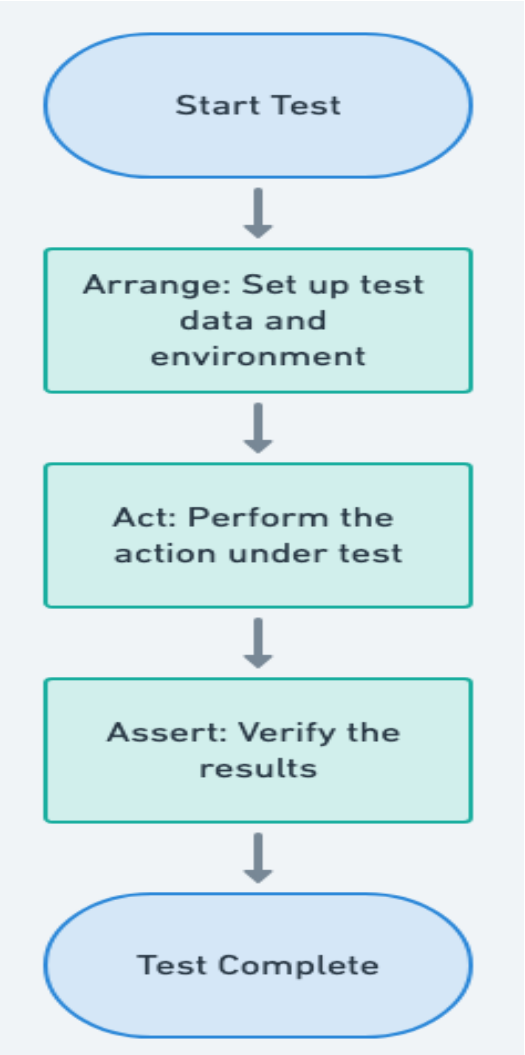
- **Unit Tests → many**
  - Fast, stable, cheap. Catch logic bugs early.
- **Widget Tests → moderate**
  - Validate UI behavior and interactions.
- **Integration Tests → few**
  - Slow & expensive. Use for critical user flows only.



Flutter Testing Workflow

General steps for any Flutter test:

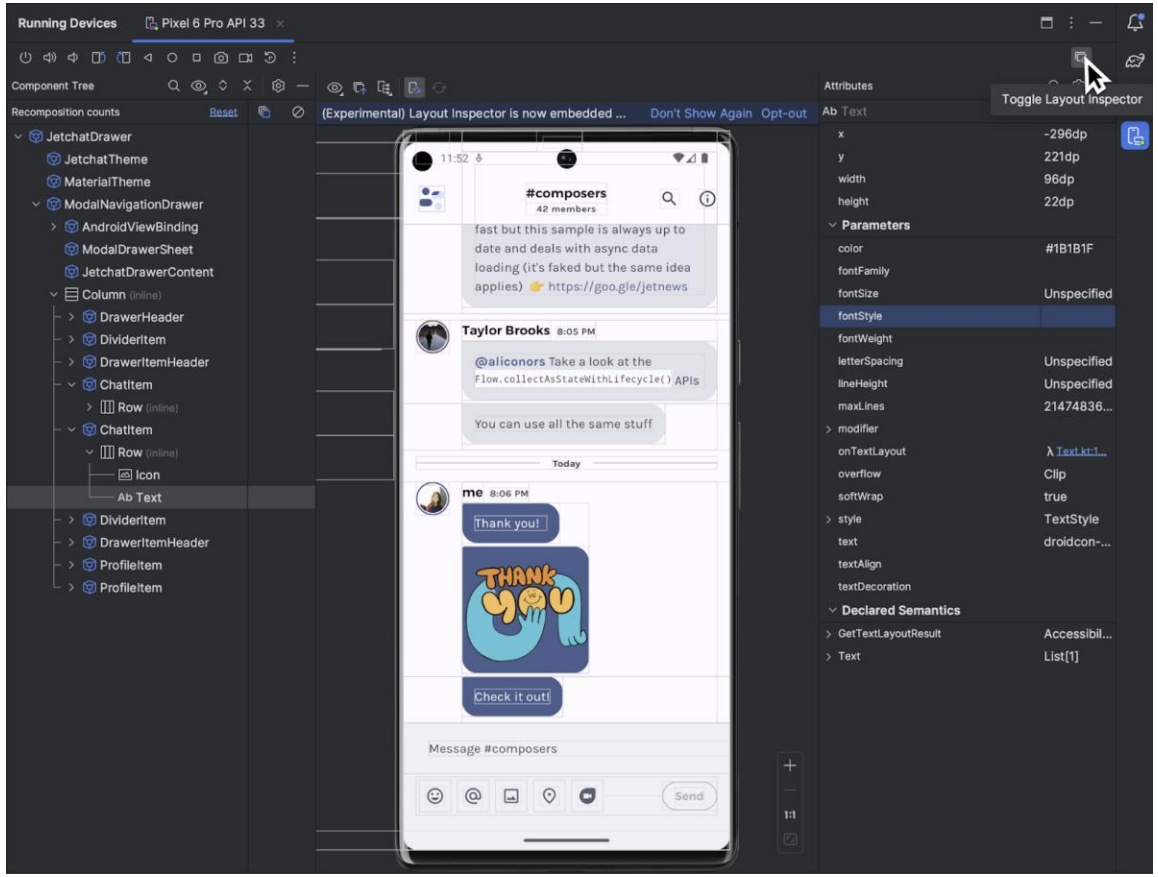
- Define the expected behavior
- Prepare the test environment
- Execute logic / render widget
- Simulate interactions
- Use expect() to verify the results



Debugging Goals in Flutter

Debugging in Flutter focuses on:

- Layout issues (constraints, overflow)
- State update failures
- Performance slowdowns
- Frequent unnecessary rebuilds
- Navigation issues



How Tests Map to Real Flutter Problems

Key Insight

- Most UI issues start from logic → unit tests catch problems early.
- Layout & interaction problems → widget tests detect what unit tests cannot see.
- Multi-screen or user-flow bugs → integration tests validate the entire experience.

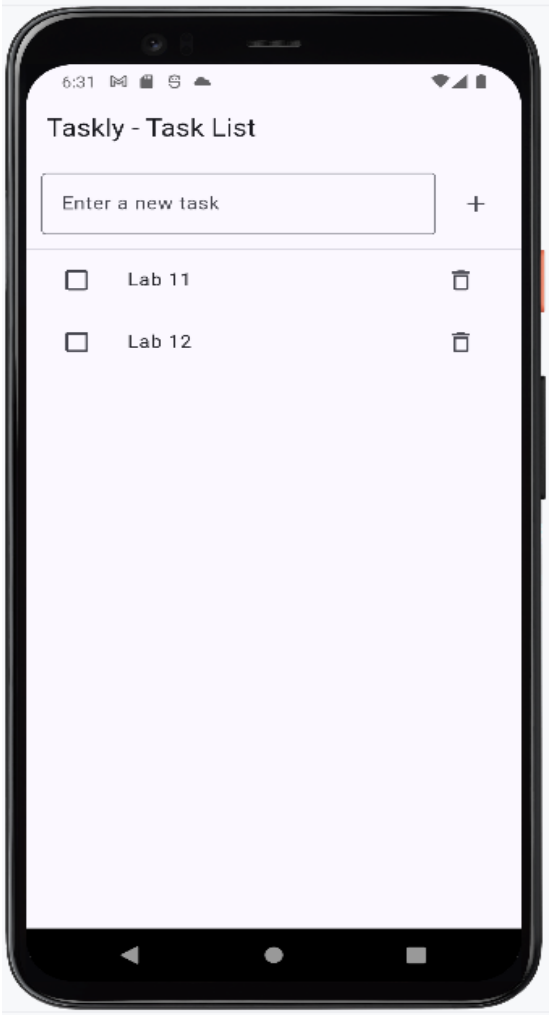
Flutter Issue	Recommended Test	Why
Logic failure	Unit test	Fast + stable
UI not updating	Widget test	Validate rebuild
Wrong navigation	Navigation test	Checks route + data
Entire feature broken	Integration test	Simulates real use

Introducing the Taskly Demo App

We use Taskly, a simple Todo App, to demonstrate each test type:

Features:

- Task List screen
- Task Detail screen
- Add, delete, toggle tasks
- Edit and save tasks



Taskly App Overview

App Structure

- TasklyApp is the application's entry point.
- Provider (TaskProvider) manages app-wide state.
- TaskRepository handles CRUD logic for tasks.
- TaskListScreen is the primary **UI used in testing.**

Why This Matters for Testing

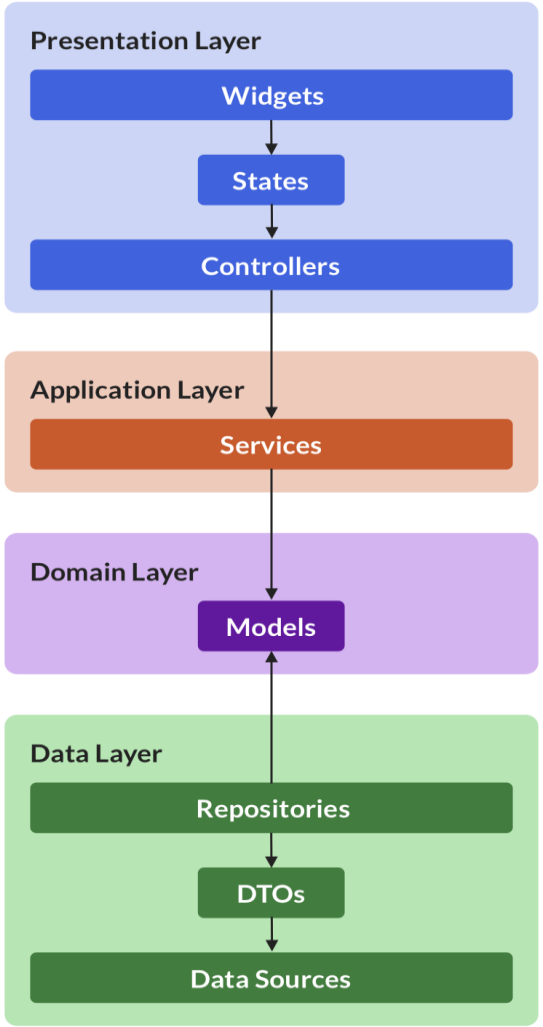
- Shows which widget is pumped during tests.
- Defines the state flow: UI → Provider → Repository.
- Provides context for unit, widget, navigation, and integration testing.

```
class TasklyApp extends StatelessWidget {
  const TasklyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (_) => TaskProvider(TaskRepository()),
      child: const MaterialApp(
        home: TaskListScreen(),
      ),
    );
  }
}
```

Demo 11.1 – Unit Test: Task Model

File: test/unit/demo_11_1_task_model_test.dart

```dart
import 'package:flutter_test/flutter_test.dart';
import 'package:demo11_taskly/models/task.dart';

void main() {
  group('Task model - Demo 11.1', () {
    test('toggle() should switch completed state', () {
      // STEP 1 - Arrange
      final task = Task(id: '1', title: 'Demo task');

      expect(task.completed, false);

      // STEP 2 - Act
      task.toggle();

      // STEP 3 - Assert
      expect(task.completed, true);

      // Optional: toggle again
      task.toggle();
      expect(task.completed, false);
    });
  });
}
```



Unit Test Demonstration (Task Model)

What we test here

- Toggle completion
- Update model state
- Logic stays predictable

Expected Output

- Initial: completed = false
- After toggle(): completed = true

Snippet

```
final task = Task(id:'1', title:'A');
task.toggle();
expect(task.completed, true);
```

Explanation

Line of Code	User Action Simulated	Purpose in the Test	What It Verifies
`final task = Task(...)`	Creates a new Task object	Arrange: prepare test data	Model starts with known state
`task.toggle();`	Toggles completion flag	Act: perform operation	Business logic changes state
`expect(task.completed, true);`	Check result	Assert: validate output	Logic behaves correctly

Demo 11.2 – Widget Test: Add Task Updates UI

File: test/widget/demo_11_2_task_list_add_test.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:provider/provider.dart';

import 'package:demo11_taskly/providers/task_provider.dart';
import 'package:demo11_taskly/repository/task_repository.dart';
import 'package:demo11_taskly/screens/task_list_screen.dart';

Widget createTaskListTestApp() {
  return ChangeNotifierProvider(
    create: (_) => TaskProvider(TaskRepository()),
    child: const MaterialApp(
      home: TaskListScreen(),
    ),
  );
}
```

```
void main() {
  group('Task List - Demo 11.2', () {
    testWidgets('adding a task should display it in the list',
        (WidgetTester tester) async {
      // STEP 1 - Arrange
      await tester.pumpWidget(createTaskListTestApp());
      expect(find.text('No tasks yet. Add one!'), findsOneWidget);

      // STEP 2 - Act
      await tester.enterText(
        find.byKey(const Key('taskTextField')),
        'Buy milk',
      );
      await tester.tap(find.byKey(const Key('addTaskButton')));
      await tester.pump();

      // STEP 3 - Assert
      expect(find.text('Buy milk'), findsOneWidget);
      expect(find.text('No tasks yet. Add one!'), findsNothing);
    });
  });
}
```

Widget Test Demonstration (Add Task Interaction)

What we test

- Enter text
- Tap “Add”
- List updates with newly added task

Expected Output

- Before: “No tasks yet. Add one!”
- After: “Buy milk”

What this proves

- UI rebuilds correctly
- Provider state updates
- ListView reacts to new data

Snippet

```
await tester.enterText(find.byKey(Key('taskTextField')), 'Buy milk');
await tester.tap(find.byKey(Key('addTaskButton')));
await tester.pump();
```

Explanation

Line of Code	User Action Simulated	Purpose in Test	What It Verifies
`enterText(..., 'Buy milk')`	User types into the input field	Act: simulate typing	The TextField accepts input
`tap(addTaskButton)`	User presses the Add button	Act: trigger UI action	Provider receives update request
`pump()`	UI rebuild	Assert preparation	The new task appears on screen

Demo 11.3 – Navigation Test: List → Detail

File : test/widget/demo_11_3_navigation_to_detail_test.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:provider/provider.dart';

import 'package:demo11_taskly/models/task.dart';
import 'package:demo11_taskly/providers/task_provider.dart';
import 'package:demo11_taskly/repository/task_repository.dart';
import 'package:demo11_taskly/screens/task_list_screen.dart';

Widget createTaskListWithSeededTask() {
  final repo = TaskRepository();
  repo.addTask(Task(id: '1', title: 'Demo task for navigation'));

  return ChangeNotifierProvider(
    create: (_) => TaskProvider(repo),
    child: const MaterialApp(
      home: TaskListScreen(),
    ),
  );
}
```

```dart
void main() {
  group('Navigation - Demo 11.3', () {
    testWidgets('tapping a task navigates to Task Detail screen',
        (WidgetTester tester) async {
      // STEP 1 - Arrange
      await tester.pumpWidget(createTaskListWithSeededTask());
      expect(find.text('Demo task for navigation'), findsOneWidget);

      // STEP 2 - Act
      await tester.tap(find.text('Demo task for navigation'));
      await tester.pumpAndSettle();

      // STEP 3 - Assert
      expect(find.text('Task Detail'), findsOneWidget);
      expect(find.byKey(const Key('detailTitleField')), findsOneWidget);
    });
  });
}
```

Navigation Test Demonstration (List → Detail)

What We Test

- Click on the item in the list
- Navigate to the detail screen
- Data passed to the detail is correct

Expected output

- New screen shows: "Task Details"
- Detail form loaded with task data

Why this is important

- Navigation errors are common
- Ensure routes and widget tree are correct

Snippet

```
await tester.tap(find.text('Demo task'));
await tester.pumpAndSettle();
expect(find.text('Task Detail'), findsOneWidget);
```

Explanation

Line of Code	User Action Simulated	Purpose in Test	What It Verifies
`tap(find.text('Demo task'))`	User taps a list item	Act: trigger navigation	Navigation event is fired
`pumpAndSettle()`	Wait for animations & navigation	Arrange for assertion	Destination screen fully loads
`expect(find.text('Task Detail')...)`	Check screen title	Assert: confirm navigation result	App navigated to the correct page

Demo 11.4 – Integration Test: Full CRUD Flow

File: test/widget/demo_11_4_integration_flow_test.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:provider/provider.dart';

import 'package:demo11_taskly/providers/task_provider.dart';
import 'package:demo11_taskly/repository/task_repository.dart';
import 'package:demo11_taskly/screens/task_list_screen.dart';

Widget createFullAppForIntegration() {
  return ChangeNotifierProvider(
    create: (_) => TaskProvider(TaskRepository()),
    child: const MaterialApp(
      home: TaskListScreen(),
    ),
  );
}

void main() {
  group('Integration flow – Demo 11.4', () {
    testWidgets('add, open detail, edit, save and return to updated list',
        (WidgetTester tester) async {
      // STEP 1 – Arrange
      await tester.pumpWidget(createFullAppForIntegration());
```

```
      // STEP 2 – Act: add a new task
      await tester.enterText(
        find.byKey(const Key('taskTextField')),
        'Original title',
      );
      await tester.tap(find.byKey(const Key('addTaskButton')));
      await tester.pump();

      // Open detail
      await tester.tap(find.text('Original title'));
      await tester.pumpAndSettle();
      expect(find.text('Task Detail'), findsOneWidget);

      // Edit title
      await tester.enterText(
        find.byKey(const Key('detailTitleField')),
        'Updated title',
      );
      await tester.tap(find.byKey(const Key('detailSaveButton')));
      await tester.pumpAndSettle();

      // STEP 3 – Assert
      expect(find.text('Updated title'), findsOneWidget);
      expect(find.text('Original title'), findsNothing);
    });
  });
}
```

Integration Test Demonstration (Full Task Flow)

What we test

- Add task
- Open detail
- Edit title
- Save
- Confirm new title appears in list

Expected Output

- Old title disappears
- New title displayed

Snippet

```
await tester.enterText(find.byKey(Key('taskTextField')), 'Original');

...

await tester.enterText(find.byKey(Key('detailTitleField')), 'Updated');

await tester.tap(find.byKey(Key('detailSaveButton')));
```

Explanation

Line of Code	User Action Simulated	Purpose in Test	What It Verifies
`enterText(taskTextField, 'Original')`	User creates a new task	Step 1 – Add task	Input field and add flow work
`enterText(detailTitleField, 'Updated')`	User edits the task title	Step 2 – Edit task	Detail screen correctly receives data
`tap(detailSaveButton)`	User saves the edited task	Step 3 – Save operation	Provider updates data + UI refresh

Best Practices

- Keep tests small and focused
- Use keys for widget tests
- Mock repositories when possible
- Avoid testing Flutter framework functionality
- Don’t overuse integration tests

Summary

In this module, students learned:

- In this module, students learned how to:
- Identify common UI and state-related issues in Flutter apps
- Apply the Flutter testing pyramid (Unit → Widget → Integration)
- Write unit tests to verify logic and model behavior
- Write widget tests to validate UI rendering and interactions
- Write integration tests to confirm full user flows
- Use DevTools to diagnose layout, rebuild, and navigation issues

References

- Mastering Flutter 2025: Learn to Develop Flutter Apps, Kevin Moore, Packt Publishing. Chapter 13, 14, 15
- Flutter Documentation – Testing: https://docs.flutter.dev/cookbook/testing
- Flutter Documentation – DevTools: https://docs.flutter.dev/tools/devtools/overview
- Provider Package Documentation: https://pub.dev/packages/provider
- flutter_test Library API Reference: https://api.flutter.dev/flutter/flutter_test/flutter_test-library.html